

FormalScience: Scalable Human-in-the-Loop Autoformalisation of Science with Agentic Code Generation in Lean

Anonymous ACL submission

Abstract

Formalising informal mathematical reasoning into formally verifiable code is a significant challenge for large language models. In scientific fields such as physics, domain-specific machinery (*e.g.* Dirac notation, vector calculus) imposes additional formalisation challenges that modern LLMs and agentic approaches have yet to tackle. To aid autoformalisation in scientific domains, we present FormalScience; a human-in-the-loop agentic pipeline that enables a single domain expert (without deep formal language experience) to produce *syntactically correct* and *semantically aligned* formal proofs of informal reasoning for low economic cost. Using the FormalScience pipeline we construct FormalPhysics, a dataset of 200 university-level (LaTeX) physics problems and solutions (primarily quantum mechanics and electromagnetism), along with their Lean4 formal representations. Compared to existing formal math benchmarks, FormalPhysics achieves perfect formal validity and exhibits greater statement complexity. We evaluate open-source models and proprietary systems on a statement autoformalisation task on our dataset via zero-shot prompting, self-refinement with error feedback, and a novel multi-stage agentic approach, and explore autoformalisation limitations in modern LLM-based approaches. We provide the first systematic characterisation of semantic drift in physics autoformalisation in terms of concepts such as notational collapse and abstraction elevation which reveals what formal language verifies when full semantic preservation is unattainable. Code available online.¹

1 Introduction

The informal mathematical reasoning produced by human researchers deviates significantly from the reasoning expressed in formal languages (FL) compiled by automated theorem provers and formal systems (Iancu and Rabe, 2011; McCarthy, 2022;

Aliyev, 2024). Formal systems provide strict validation of mathematical statements and proofs at the cost of requiring strict adherence to the syntax of the type-theoretic FL and mathematical library of the prover. This particularly creates barriers towards their broader formalisation in the quantitative sciences (Kaliszyk et al., 2015; Meadows and Freitas, 2021; Bobbin et al., 2023).

The emergence of large language models (LLMs) has partially provided the substrate within which the gap between informal and formal reasoning may be traversed (*i.e.* autoformalisation (Wu et al., 2022)). However, LLMs have shown increasing hallucination rates with greater task complexity (Opedal et al., 2024; Li et al., 2025) or under out-of-distribution shifts (Stolfo et al., 2022; Meadows et al., 2025), which leads a semantic and syntactic gap when transferring informal representations to formal representations (Zhang et al., 2024; Ganguly et al., 2025). As the problem space deviates further from the mathematics most compatible with the FL, the harder it is to build robust LLM-based autoformalisation methods (Zhang et al., 2025c). These critical limitations inhibit the automation of scientific verification, exploration and fact-checking (Lu et al., 2024a; Yamada et al., 2025), making autoformalisation especially challenging in natural sciences such as *Physics*, where semantic drift is significantly magnified by solver incompatibilities with domain notation and calculus.

To understand the bottlenecks of autoformalisation within scientific domains, we:

- (i) Introduce FormalScience (Figure 1): a lightweight, efficient, and cost-effective human-in-the-loop pipeline for scientific statement-proof generation and semi-autoformalisation, which can convert informal reasoning into largely aligned Lean4 code with a 100% formal guarantee. The pipeline is also scalable, enabling the generation of high-quality formal datasets in scientific domains for fine-tuning or evaluating AI systems.

¹anonymous github link (soon)

Key Inputs

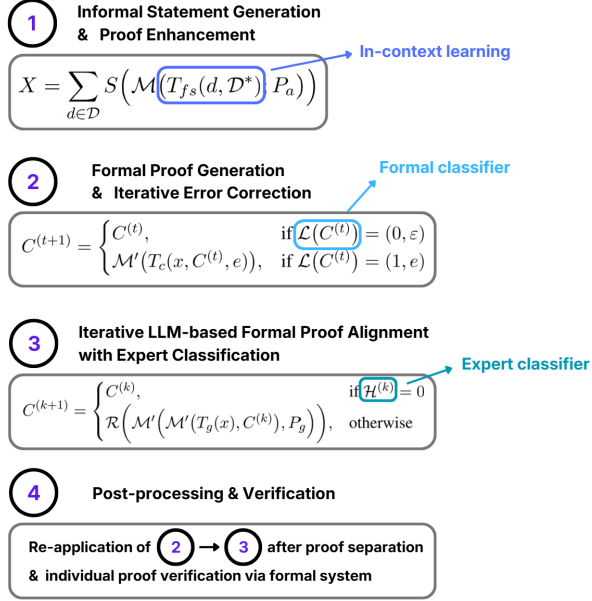
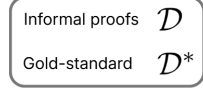


Figure 1: An overview of the FormalScience approach.

(ii) Present FormalPhysics: a corpus generated from the FormalScience pipeline comprising of 200 university-level physics statements, informal LaTeX proofs, and formal (Lean4) code. Evaluation using LLM judges confirms that FormalPhysics *matches or exceeds* canonical datasets in formal validity, formal quality, and complexity.

(iii) Evaluate open-source (up to 30B parameters) and proprietary models on performing autoformalisation of statements in FormalPhysics with three increasingly complex LLM-based approaches: zero-shot autoformalisation, self-refinement with error feedback, and a novel neuro-symbolic multi-stage agentic approach. The leading methodology (GPT-OSS-20B agent) achieves a formal validity of 31% and competitive alignment scores in statement autoformalisation. All other approaches are subject to a distinct alignment-validity trade-off.

(iv) Provide a detailed qualitative and quantitative analysis of alignment drift in FormalPhysics by characterising drift categories, and using each characterisation to quantitatively describe what Lean4 actually verifies (per Physics subdomain) when alignment drift exists yet the formal code successfully compiles, and what formalisation guarantees arise as a consequence. This serves as a step towards explainable verification for Science.

2 FormalScience: A human-in-the-loop agentic semi-autoformalisation pipeline

Autoformalisation refers to the automatic conversion of an informal content (e.g. LaTeX) into formally verifiable code, that can be compiled by a solver or formal system (e.g. Lean). However, producing syntactically correct and semantically aligned formalisations without human involvement remains a significant challenge (as we later demonstrate). We propose FormalScience (Alg. 1), a semi-automatic pipeline for generating problem statements, expanded informal solutions, and formally valid code in a scientific domain, thereby facilitating the study of pure autoformalisation.

We assume a collection of informal proofs $\mathcal{D}$ (e.g. LaTeX derivations) and a gold-standard set of informal statement and proofs, $\mathcal{D}^* = [(\mathcal{S}_1, \mathcal{P}_1), \dots, (\mathcal{S}_{N'}, \mathcal{P}_{N'})]$ (e.g. $N' = 5$). Using in-context learning, a *few-shot template* (T_{fs}) is formulated (see Appendix B) for the purpose of (1) generating statements which correspond to the informal proofs in $\mathcal{D}$; and (2) altering the proofs in $\mathcal{D}$ to align with the gold-standard $\mathcal{D}^*$. This may for instance include the addition of fine-grained derivation steps or textual context. We can write the resulting intermediate dataset as array concatenation

$$X = \sum_{d \in \mathcal{D}} S(\mathcal{M}(T_{fs}(d, \mathcal{D}^*); P_a)) \quad (1)$$

where d is a batch of informal proofs, $\mathcal{M}$ is a multi-turn LLM prompting session, and S is a post-processing function splitting the LLM output $\mathcal{M}(T_{fs}(d, \mathcal{D}^*))$ into separate informal statement-proof pairs, such that $x = [(\mathcal{S}_1, \mathcal{P}_1), \dots, (\mathcal{S}_{B'}, \mathcal{P}_{B'})]$ (where B' is the batch size, see Alg 1), and $x \in X$ aligns with gold-standard $\mathcal{D}^*$. The fixed prompt P_a is used to assess alignment in the prompting session, where the expert suggests improvements and verifies alignment before the next stage.

Next, to obtain the formal code C for each statement-proof pair, we rely on Lean 4 and its Mathlib toolchain to iteratively attempt compilation and return any fatal error messages. We define the tool $\mathcal{L}$ (i.e. Lean compiler) such that:

$$\mathcal{L}(C) = \begin{cases} (0, \varepsilon), & \text{if } C \text{ compiles} \\ (1, e), & \text{otherwise} \end{cases} \quad (2)$$

where e is the error message for a given iteration, and ε is the empty string. To begin iterative error

correction, we initially use a code generation template T_g to prompt an LLM-based agent $\mathcal{M}'$ to use tool $\mathcal{L}$, with correction template T_c . If we define the first output as $C^{(0)} = \mathcal{M}'(T_g(x))$, we obtain compilable code via

$$C^{(t+1)} = \begin{cases} C^{(t)}, & \text{if } \mathcal{L}(C^{(t)}) = (0, \varepsilon) \\ \mathcal{M}'(T_c(x, C^{(t)}, e)), & \text{if } \mathcal{L}(C^{(t)}) = (1, e) \end{cases} \quad (3)$$

which terminates when $t^* = \min\{t : \mathcal{L}(C^{(t)}) = (0, \varepsilon)\}$, to give $C = C^{(t^*)}$. In practice, the multi-turn conversational agent $\mathcal{M}'$ operates over a representation of the chat history contained within its context window. We can write this correction loop in shorthand as $C = \mathcal{R}(C^{(0)})$.

The previous phase does not consider whether semantic drift occurs between the informal and formal representations. Defining the LLM-based evaluation of the formal code's alignment with the initial prompt ($T_g(x)$) as $\mathcal{M}'(T_g(x), C)$, we can write the *expert alignment classification* step at iteration k as

$$\mathcal{H}^{(k)} = \mathcal{H}(\mathcal{M}'(T_g(x), C^{(k)})) \in \{0, 1\} \quad (4)$$

where the human intervenes as a binary classifier (analogous to compilation tool $\mathcal{L}(C)$). We obtain both *aligned* and *corrected* code via the following iterative process

$$C^{(k+1)} = \begin{cases} C^{(k)}, & \text{if } \mathcal{H}^{(k)} = 0 \\ \mathcal{R}(\mathcal{M}'(\mathcal{M}'(T_g(x), C^{(k)}), P_g)), & \text{otherwise} \end{cases} \quad (5)$$

which terminates when $k^* = \min\{k : \mathcal{H}^{(k)} = 0\} \in [0, \mathcal{P}]$, where patience $\mathcal{P}$ denotes the maximum number of expert alignment classification iterations, the fixed prompts P_a and P_g are respectively used provide an LLM-based assessment of alignment and generate improvements, and the code is iteratively corrected via $\mathcal{R}$ (Eq. 3).

Finally, a post-processing step is used to extract individual proofs from each output generated by Eq. 5, which results in tuples (S, P, C) . However, the post-processing may have introduced errors, so all C are recompiled with $\mathcal{L}$ to find invalid formal proofs, which are iteratively improved via Eq. 3-5.

3 FormalPhysics: Physics Formalization in Lean4

We select physics as the target scientific domain and Lean as the target formal language, and apply

FormalScience using GPT-5.1 and Claude-Opus-4.5 (Anthropic, 2025a). The resulting dataset (FormalPhysics) is a corpus containing 200 physics statements, informal LaTeX proofs, and complete formal proofs. The examples used as input to FormalScience to generate FormalPhysics are sourced from related work (Meadows et al., 2024), and the human-in-the-loop pipeline was conducted by one physics expert within one month. With a motivated group of experts using FormalScience, they could generate thousands of verified *field-specific* formalisations from scientific works in a similar timeframe. Such data could be used to fine-tune custom LLMs and improve or evaluate the capabilities of state-of-the-art mathematical discovery approaches. The exact implementation of Alg. 1 used to construct FormalPhysics is described in Appendix A.

3.1 Benchmark Comparison

We compare the properties of our dataset with existing benchmarks for formal mathematics. All datasets in Table 1 contain a formal language statement (s_{FL}) (e.g. Lean4 statement), which is the formal representation of a given natural language statement (s_{NL}). *Statement Autoformalisation* is the task of automatically translating the NL statement to the FL statement ($s_{\text{NL}} \rightarrow s_{\text{FL}}$), yet only 7/8 datasets contain an NL statement. A natural language proof (p_{NL}) is an informal proof of a given NL statement. Automating this reasoning ($s_{\text{NL}} \rightarrow p_{\text{NL}}$) is typical of *scientific/mathematical QA tasks*, yet only 5/8 datasets contain an informal NL proof (e.g. LaTeX derivation). Only 3/8 datasets contain full FL proofs (p_{FL}) (e.g. without "sorries"), thereby supporting *automated theorem proving* and *full theorem autoformalisation* tasks. Only FormalPhysics and Herald-Proof are compatible with every task.

We further randomly sample 200 examples from each benchmark and evaluate them across different dimensions. We use elements from a taxonomy for autoformalisation evaluation (Zhang et al., 2025b) to evaluate samples. Through this method, both the *formal aspects* of the generated formal code and its *alignment* with the input natural language (NL) statement are properly evaluated.

Evaluation methodology. For a given input example, each element can be measured via a *binary classification* with respect to a target characteristic. Classification results are averaged over the sample

Dataset	Size	Domain	s_{NL}	p_{NL}	s_{FL}	p_{FL}
miniF2F (Zheng et al., 2022)	488	Olympiad (Ol) Math	✓	✓	✓	Partial
ProofNet (Azerbaiyev et al., 2023a)	371	Undergraduate (UG) Math	✓	✓	✓	×
Lean-Dojo (Yang et al., 2023a)	98,734	Mathlib	×	×	✓	✓
Lean Workbook (Ying et al., 2024)	57,231	High-School Math	✓	×	✓	Partial
FormalMATH (Yu et al., 2025)	5,560	Ol & UG Math	✓	✓	✓	×
Herald-Statement (Gao et al., 2025)	579,883	Mathlib	✓	×	✓	×
Herald-Proof (Gao et al., 2025)	44,553	Mathlib	✓	✓	✓	✓
FormalPhysics	200	Advanced Physics	✓	✓	✓	✓

Table 1: Properties of Lean4 formal benchmarks. s_{NL} : Natural Language Statement; p_{NL} : Natural Language Proof; s_{FL} : Formal Language Statement; p_{FL} : Formal Language Proof.

Dataset	s_{NL} Complexity		s_{FL} Correctness		$s_{NL}-s_{FL}$ Alignment	
	Objects	Formulae	FV (%)	FQ (%)	LP (%)	MC (%)
miniF2F	3.14±1.55	3.21±1.53	88.00	63.00	92.00	92.00
ProofNet	3.67±1.48	3.62±1.52	95.50	61.50	77.50	77.50
Lean Workbook	3.67±1.99	3.62±2.26	89.00	46.00	78.00	85.00
FormalMATH	4.47±2.45	4.53±2.62	97.50	80.00	98.00	96.50
Herald-Statement	4.92±2.43	4.80±2.30	80.50	63.50	87.00	87.00
Herald-Proof	6.57±2.32	6.42±2.37	2.00	73.00	94.50	94.00
FormalPhysics	6.41±2.34	6.22±2.13	100.00	73.50	72.00	72.50

Table 2: Statistics derived from 200 examples randomly selected from each dataset. **Objects**: How many math or physics objects excluding explicit numbers and variables are mentioned directly in the natural language statement? **Formulae**: How many math or physics formulae are mentioned directly in the natural language statement? **FV**: Formal Validity (*i.e.* Pass rate); **FQ**: Formal Quality; **LP**: Logical Preservation; **MC**: Mathematical Consistency.

to give the percentages in Table 2. **Formal Validity** (FV) is judged by the Lean4 theorem prover, while the remaining metrics are assessed via LLM-as-a-judge. **Formal Quality** (FQ) is the rate that the formal code is of high quality in regard to structural clarity and usefulness. **Logical Preservation** (LP) is the rate that the code captures the logical structure and content of the original NL statement. **Mathematical Consistency** (MC) is the rate that the formal code accurately represents mathematical objects and operations present in the NL statement. We also measure the number of objects and formulae in the natural language statements with LLM judges. We prompt GPT-4.1-mini with 0.2 temperature to obtain judgments. Although there might be some noise in the judgments, the underlying LLM is still unbiased towards a specific benchmark and results are still indicative.

Formalisation in physics is more complex. A given natural language statement in FormalPhysics contains on average *twice as many mathematical objects and formulae* as miniF2F (Zheng et al., 2022), ProofNet (Azerbaiyev et al., 2023b), and Lean Workbook (Ying et al., 2024). FormalPhysics also contains $\approx 33\%$ more objects/formulae than the recent FormalMath (Yu et al., 2025) and Herald-

Statement (Gao et al., 2025) datasets per example, matched only by Herald-Proof (Table 2).

High formal validity with examples generated through FormalScience. All formal code examples generated from our pipeline are syntactically valid on the latest Lean4 version (see FV, Table 2). Notably, miniF2F and ProofNet were released circa 2023 yet respectively score 88% and 96% successful compilation rates (on an older version of Lean). Herald-Proof (Gao et al., 2025) achieves only 2% FV, while FormalMATH (Yu et al., 2025) scores 98%. FormalPhysics obtains a perfect score.

High formal quality (FQ), low logical preservation (LP) and mathematical consistency (MC). FormalPhysics obtains the second highest score for formal quality (a reference-free evaluation of formal code quality), while scoring the lowest for both logical preservation and mathematical consistency with the NL statement. This is a natural consequence stemming from the fact that around half of all NL statements within FormalPhysics contain vector calculus (*i.e.* electromagnetism) or Dirac notation (*i.e.* quantum mechanics) with non-commutative operators. Lean4 does not directly support vector calculus or Dirac notation, and strug-

LLM	FV (%)	FQ (%)	LP (%)	MC (%)
(1) Zero-Shot Autoformalisation				
Qwen2.5-Coder-7B	1.00	15.00	24.00	20.50
DeepSeek-Prover-7B	13.00	23.00	27.50	24.00
Kimina-7B	51.50	6.50	10.50	9.50
GPT-OSS-20B	4.50	68.50	73.00	72.50
GPT-5.1	14.50	79.50	76.50	77.00
(2) Self-Refinement with Error Feedback				
Qwen2.5-Coder-7B	1.00	16.50	23.00	19.50
DeepSeek-Prover-7B	4.50	17.00	23.00	23.00
Kimina-7B	23.00	6.50	9.50	8.00
GPT-OSS-20B	7.50	70.50	77.00	79.00
GPT-5.1	17.00	82.50	82.00	82.00
(3) Agentic Code Generation Pipeline				
Qwen3-Sonnet-14B	52.00	1.00	10.50	6.50
GPT-OSS-20B	31.00	73.00	72.50	73.00
Qwen3-Coder-30B	5.50	49.50	59.00	48.00
(4) FormalScience (ours)				
GPT-5.1 / Claude-4.5	100.00	73.50	72.00	72.50

Table 3: Performance of LLM-based approaches on the FormalPhysics corpus.

gles with basic derivatives and integrals (Bobbins et al., 2023), hence alternative strategies and notation were required to successfully compile the code. Formalising physics is extremely challenging (Kaliszyk et al., 2015) and we qualitatively explore such misalignment in Section 5.

4 Experimental Results

Towards fully automated and formally verifiable scientific reasoning, we use FormalPhysics for statement autoformalisation with both open-source and proprietary LLMs in three increasingly complex inference pipelines: (1) zero-shot prompting; (2) self-refinement with error feedback; (3) an agentic code generation approach based on a recent framework (Wang et al., 2024). The results are provided in Table 3.

Models. We use open-source Qwen2.5-Coder-7B (Hui et al., 2024), DeepSeek-Prover-V2-7B (Ren et al., 2025), Kimina-Autoformalizer-7B (Wang et al., 2025a), GPT-OSS-20B (OpenAI et al., 2025), a distillation of Claude-Sonnet-4.5 (Anthropic, 2025b) onto Qwen3-14B (TeichAI / Liantix, 2025; Yang et al., 2025a), Qwen3-Coder-30B, and *closed-source* GPT-5.1 (OpenAI, 2025).

4.1 Zero-shot and self-refinement pass

We discuss physics autoformalisation baselines for settings (1) and (2) (prompts in Appendix B). The self-refinement baselines are based on the zero-shot

formalisations from the same LLM.

Trade-off between formal validity and semantic alignment. The Spearman and Pearson coefficients of FV and the mean of FQ, LP, and MC are both zero to one decimal place (with $p > 0.9$), indicating that an approach with high probability of generating syntactic valid formalisations for physics is hard to capture the intended semantics at same time.

Invariance to naive prompting. The LLM-as-a-judge scores (FQ, LP, MC) are effectively unchanged per model between the zero-shot and self-refinement settings. The error-based self-refinement method uses error details to improve zero-shot output, at the cost of $\approx 2x$ the token usage, without clear improvement to formal validity or alignment scores for FormalPhysics.

4.2 Agentic code generation

To establish a best-effort open-source baseline on consumer-grade hardware, we implement an agentic code generation pipeline system aiming to maximise compilation rates and alignment without human intervention. We provide a full derivation of our implementation and further details in Appendix C and Alg. 2. Each baseline in Table 3 required **100+ hours of compute on a 5090 RTX GPU** (*i.e.* 30+ minutes per Physics proof). This is approximately equivalent to the code generation rate of the FormalScience approach.

We use an LLM as the base model for a CodeAgent within the smolagents framework (Wang et al., 2024). The agent may use Python functions as tools during inference (in a ReAct (Yao et al., 2022) cycle) which generally features a planning step, a tool-calling action step, and an observation step where the model assesses the tool’s output. The agent may output a final answer based on the observation or begin another cycle.

Our implementation features two primary stages. First, an initial generation phase outputs Lean code which is fed to a surface guard that rejects code containing forbidden tokens, incomplete proofs, or malformed imports before compilation. Second, an iterative correction phase compiles the code and categorises errors. *Structural* errors (syntax, unknown identifiers, missing modules) trigger full regeneration using the base LLM with hints based on error type, while *semantic* errors (type mismatches, unsolved goals) are addressed using a patch agent

that applies minimal unified diffs. It terminates after 25 correction iterations (full ReAct cycles) or successful compilation in Lean.

Open-source models can overcome the alignment-validity trade-off. The previous GPT-OSS-20B baselines can only produce less than 10% formally valid formalisations. The agentic approach improved this to 31% without any significant decrease in LLM-as-a-judge scores.

Autoformalisation is an emergent capability dependent upon parameter count, neuro-symbolic integration, and test-time scaling. FormalScience obtained a formal validity score of over 3x the best open-source agentic approach. Furthermore, the results are highly sensitive to the base LLM choice, where larger models (e.g. 14B, 30B) do not necessarily outperform the naive prompting approaches utilising smaller models (e.g. Kimina-7B). *Without iterative dialogue with a symbolic prover*, it is difficult for a leading transformer-based LLM to produce physics formalisations with high formal validity, regardless of the test-time scaling techniques used. Similarly, there exists a minimum base LLM reasoning capability (determined by parameter count, context window, etc.) required to make effective use of symbolic tools (e.g. minimal unified diff) in agentic pipelines. Without any/minimal test-time scaling (e.g. zero-shot) GPT-5.1 obtained only 15% formal validity, yet when used within FormalScience it performed significantly better. These three extremes (no symbolic tools, low base LLM intelligence, no test-time scaling) demonstrate the type of experimental optimisation problem required to deliver science autoformalisation pipelines.

5 Alignment and Qualitative Analysis

When a syntactically valid formalisation uses fundamentally different mathematical objects, *what exactly has been verified?* What partial guarantees can formalisation provide when full semantic preservation is unattainable? To answer this we quantitatively measure alignment divergence using a distinct categorisation schema, and use this to guide a qualitative characterisation of the semantic drift induced by formalisation to Lean. We define the following drift categories (visualised in Fig. 2):

Notational Collapse: Domain-specific physics no-

tation collapsed to simpler mathematical objects.

The relevant Fig. 2 example expects a solution integrating over the continuous delta function and multiple substitution operations with terms defined using Dirac/braket notation. The Lean proof correctly defines x as a real scalar but inappropriately defines the quantum state *vector* $|\Psi\rangle$ as the complex scalar Ψ . Quantum mechanics is formalised within a complete, complex inner-product (i.e. Hilbert) space, which is enforced implicitly through Dirac notation. When $|\Psi\rangle$ is collapsed to Ψ in this manner (i.e. ignoring $\langle x|\Psi\rangle = \Psi(x)$) the fundamental formalism of QM is not respected.

What did Lean verify? Essentially $z \cdot z^* = |z|^2$ for $z \in \mathbb{C}$. All quantum mechanical formalism (and calculus) is absent.

Abstraction Elevation: Symbolic operations replaced by abstract algebraic properties.

The Fig. 2 example requires the evaluation of a definite line integral in 3-dimensional space given the assumption that the scalar potential $\phi(r) = 0$ as $r \rightarrow \infty$. The Lean proof correctly defines a real function ϕ with real scalars q (charge) and r (radius), but skips all vector calculus.

What did Lean verify? That $x - 0 = x$ for $x \in \mathbb{R}$. The logic is that $U = q\phi(r)$ (goal statement), so the potential difference between a charge at r and at ∞ is $\Delta U = q\phi(r) - q\phi(\infty) = q\phi(r) - 0 = q\phi(r)$. The physics has been abstracted into the hypothesis statement given in the question.

Proof Strategy Substitution: Theorem proved via alternative approach to the informal derivation.

The informal solution (Fig. 2b) applies a direct partial differentiation operator to a complex function (i.e. a single operation). Lean uses an alternative (correct) strategy to show that if two functions are equal their derivatives must be equal.

What did Lean verify? The target statement $\frac{\partial^2 v}{\partial x^2} = -\frac{\partial^2 u}{\partial x \partial y}$, but circumvented direct differentiation of the supporting premise.

Implicit Premise Selection: Assumptions that are unstated in the NL statement or derivation, yet are explicitly defined as FL hypotheses.

The question requires the differentiation of a Lagrangian via the chain rule. Lean provides a deeper proof by surfacing a number of implicit premises (e.g. $1 + y' > 0 \forall y \in \mathbb{R}$, do not divide by zero). This reveals the hidden logical structure

Figure 2: Examples sourced from the FormalPhysics corpus generated via the FormalScience approach applied to Physics. Each row is associated with a different class of semantic drift (e.g. Notational Collapse, Abstraction Elevation). See Tab. 6 for a detailed analysis and additional examples.

7

6 Related Work

Recent advances in large language models (LLMs) have renewed interest in autoformalisation, which aims to bridge informal mathematical language and formal proof systems (Wu et al., 2022; Yang et al., 2025b; Mensfelt et al., 2025; Zhang et al., 2025a). Prior work has demonstrated the feasibility of translating natural-language mathematical statements into formal representations and proofs. Autoformalisation has been applied to a range of tasks, including verification of natural-language explanations in natural language inference (Quan et al., 2024b,a) and the construction of automated theorem proving pipelines (Jiang et al., 2023; Tarach et al., 2024; Liu et al., 2025b). Recent systems increasingly incorporate retrieval-augmented generation, which improves correctness and consistency by leveraging existing formal libraries, in both Isabelle (Zhang et al., 2024) and Lean (Yang et al., 2023b; Liu et al., 2025a; Wang et al., 2025b; Zhang et al., 2025d). Complementary approaches include process-driven frameworks that structure the formalization pipeline (Lu et al., 2024b) and self-consistency methods for selecting high-quality outputs (Li et al., 2024).

On the benchmarking side, existing datasets exhibit a trade-off between quality and scale. Human-curated benchmarks (Zheng et al., 2022; Azerbayev et al., 2023a; Tsoukalas et al., 2024; Poiroux et al., 2025) offer high-quality annotations but are too small for large-scale training. Benchmarks derived from formal libraries (Yang et al., 2023a; Zhang et al., 2024; Xin et al., 2025) scale better but risk data contamination due to overlap with model pretraining. Automated data generation pipelines (Jiang et al., 2024; Ying et al., 2024; Yu et al., 2025; Gao et al., 2025; Liu et al., 2025c) address scalability but often produce shallow or low-quality formalizations. Our work targets this gap by improving the quality of autoformalisation data while preserving scalability and minimizing contamination.

7 Conclusion

For the purpose of exploring autoformalisation limitations in science, we propose a human-in-the-loop agentic methodology (FormalScience) for formalising scientific reasoning in Lean, and apply it to produce a dataset (FormalPhysics) comprising 200 questions, informal solutions, and formal Physics proofs across quantum mechanics, electro-

magnetism, and other subdomains. We use compilation success rates and LLM-as-a-judge metrics to compare the formal quality and alignment of formalised statements across several notable autoformalisation benchmarks, and find FormalPhysics leads with 100% formal verification rate (and is competitive in other metrics) at the cost of alignment drift. We argue alignment issues are due to the incompatibility of formal systems with domain-specific machinery such as vector calculus and Dirac notation.

We use FormalPhysics to test the Physics autoformalisation and question-answering capabilities of three increasingly complex LLM-based inference pipelines, including zero-shot, error-driven self-refinement, and full agentic code generation involving notation surface guards, prompt regeneration based on categorised Lean compilation errors, and iterative self-refinement via a patch agent utilising the ReAct (Yao et al., 2022) framework. We test GPT-5.1 and various open-source LLMs (up to 30B parameters) on a consumer-grade GPU to estimate the gap between open-source agentic methodologies and leading human-in-the-loop autoformalisation pipelines. The prevailing open-source agent (using GPT-OSS-20B) obtained a compilation rate $\approx 1/3$ that of the FormalScience approach.

We characterise semantic formalisation errors by defining alignment drift categories, explore errors qualitatively, then determine what verification guarantees can be made depending on the specific drift type supported by domain-specific quantitative analysis. Despite the perfect formal verification rate, notational collapse occurs in most QM proofs, which instead guarantees the verification of surrogate solutions involving simple mathematical objects. Abstraction elevation guarantees meaningful physics computation is circumvented with oversimplified proofs of goal formulae, or is abstracted away with general algebraic proofs which does not necessarily respect the scientific context. We also find drift can be beneficial. In the case of implicit premise selection, without any other drift types, proofs are enriched by supporting premises and rigorous argumentation. Overall, we believe our work will support the development of LLM-based alignment metrics, guide the construction of fully automated formalisation agents, and accelerate scientific exploration.

Limitations

Dataset Scale and Scope. FormalPhysics comprises 200 examples focused on quantum mechanics and electromagnetism at university level. This scope, while sufficient for benchmark evaluation, limits generalisability to other physics subdomains (e.g., statistical mechanics, general relativity) and other sciences.

Formal System Constraints. As analysed in Section 5, Lean4’s Mathlib lacks native support for vector calculus and Dirac notation, necessitating semantic drift in formalisations. The formal proofs therefore verify algebraic consistency within abstract structures rather than the complete physical derivations. Addressing this limitation requires either extending Mathlib’s physics coverage or developing physics-specific formal libraries.

Resource Requirements. The FormalScience pipeline required approximately one month of expert effort. The agentic baselines required over 100 hours of GPU compute per 200 examples on consumer hardware. These costs may limit broader adoption and scaling to larger corpora.

Evaluation Methodology. Alignment metrics rely on LLM-as-a-judge evaluation (GPT-4.1-mini), which may not capture all dimensions of semantic preservation. The drift categorisation taxonomy we propose is one principled decomposition but not necessarily complete or unique.

Temporal Validity. Results reflect specific LLM versions and Lean4/Mathlib configurations. Model capabilities and library coverage evolve rapidly, and our findings should be interpreted in this context.

References

- Yagub N Aliyev. 2024. Set theory and elementary algebra in lean 4 theorem prover. In *2024 IEEE 18th International Conference on Application of Information and Communication Technologies (AICT)*, pages 1–6. IEEE.
- Anthropic. 2025a. Introducing claude opus 4.5. <https://www.anthropic.com/news/claude-opus-4-5>. Accessed: 2026-01-01.
- Anthropic. 2025b. Introducing claude sonnet 4.5. <https://www.anthropic.com/news/claude-sonnet-4-5>. Accessed: 2026-01-01.
- Zhangir Azerbayev, Bartosz Piotrowski, Hailey Schoelkopf, Edward W. Ayers, Dragomir Radev, and Jeremy Avigad. 2023a. [Proofnet: Autoformalizing](#)

[and formally proving undergraduate-level mathematics](#). *Preprint*, arXiv:2302.12433.

- Zhangir Azerbayev, Hailey Schoelkopf, Keiran Paster, Marco Dos Santos, Stephen McAleer, Albert Q Jiang, Jia Deng, Stella Biderman, and Sean Welleck. 2023b. Llemma: An open language model for mathematics. *arXiv preprint arXiv:2310.10631*.

- Maxwell P. Bobbin, Samiha Sharlin, Parivash Feyzishendi, An Hong Dang, Catherine M. Wraback, and Tyler R. Josephson. 2023. [Formalizing chemical physics using the lean theorem prover](#). *Preprint*, arXiv:2210.12150.

- Debargha Ganguly, Vikash Singh, Sreehari Sankar, Biyao Zhang, Xuecen Zhang, Srinivasan Iyengar, Xiaotian Han, Amit Sharma, Shivkumar Kalyanaraman, and Vipin Chaudhary. 2025. Grammars of formal uncertainty: When to trust llms in automated reasoning tasks. *arXiv preprint arXiv:2505.20047*.

- Guoxiong Gao, Yutong Wang, Jiedong Jiang, Qi Gao, Zihan Qin, Tianyi Xu, and Bin Dong. 2025. [Herald: A natural language annotated lean 4 dataset](#). In *The Thirteenth International Conference on Learning Representations*.

- Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, Kai Dang, Yang Fan, Yichang Zhang, An Yang, Rui Men, Fei Huang, Bo Zheng, Yibo Miao, Shangaoran Quan, and 5 others. 2024. [Qwen2.5-coder technical report](#). *Preprint*, arXiv:2409.12186.

- Mihnea Iancu and Florian Rabe. 2011. Formalising foundations of mathematics. *Mathematical Structures in Computer Science*, 21(4):883–911.

- Albert Q. Jiang, Wenda Li, and Mateja Jamnik. 2024. [Multi-language diversity benefits autoformalization](#). In *Advances in Neural Information Processing Systems*, volume 37, pages 83600–83626. Curran Associates, Inc.

- Albert Qiaochu Jiang, Sean Welleck, Jin Peng Zhou, Timothee Lacroix, Jiacheng Liu, Wenda Li, Mateja Jamnik, Guillaume Lample, and Yuhuai Wu. 2023. [Draft, sketch, and prove: Guiding formal theorem provers with informal proofs](#). In *The Eleventh International Conference on Learning Representations*.

- Cezary Kaliszyk, Josef Urban, Umair Siddique, Sanaz Khan-Afshar, Cvetan Dunchev, and Sofiene Tahar. 2015. Formalizing physics: automation, presentation and foundation issues. In *International Conference on Intelligent Computer Mathematics*, pages 288–295. Springer.

- Sirui Li, Wangyue Lu, Xiaorui Shi, Ke Weng, Haozhe Sun, Minghe Yu, Tiancheng Zhang, Ge Yu, Hengyu Liu, and Lun Du. 2025. Msc-180: A benchmark for automated formal theorem proving from mathematical subject classification. *arXiv preprint arXiv:2512.18256*.

721	Zenan Li, Yifan Wu, Zhaoyu Li, Xinming Wei, Xian Zhang, Fan Yang, and Xiaoxing Ma. 2024. Autoformalize mathematical statements by symbolic equivalence and semantic consistency . In <i>The Thirty-eighth Annual Conference on Neural Information Processing Systems</i> .	776
722		777
723		778
724		779
725		780
726		
727	Qi Liu, Xinhao Zheng, Xudong Lu, Qinxiang Cao, and Junchi Yan. 2025a. Rethinking and improving autoformalization: towards a faithful metric and a dependency retrieval-based approach . In <i>The Thirteenth International Conference on Learning Representations</i> .	781
728		782
729		783
730		784
731		785
732		786
733	Qi Liu, Xinhao Zheng, Renqiu Xia, Qinxiang Cao, and Junchi Yan. 2025b. Bootstrapping hierarchical autoregressive formal reasoner with chain-of-proxy-autoformalization . In <i>The Thirty-ninth Annual Conference on Neural Information Processing Systems</i> .	787
734		788
735		789
736		
737		
738	Xiaoyang Liu, Kangjie Bao, Jiashuo Zhang, Yunqi Liu, Yuntian Liu, Yu Chen, Yang Jiao, and Tao Luo. 2025c. Atlas: Autoformalizing theorems through lifting, augmentation, and synthesis of data . <i>Preprint</i> , arXiv:2502.05567.	790
739		791
740		792
741		793
742		794
743	Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. 2024a. The ai scientist: Towards fully automated open-ended scientific discovery. <i>arXiv preprint arXiv:2408.06292</i> .	795
744		796
745		
746		
747	Jianqiao Lu, Yingjia Wan, Zhengying Liu, Yinya Huang, Jing Xiong, Chengwu Liu, Jianhao Shen, Hui Jin, Jipeng Zhang, Haiming Wang, Zhicheng Yang, Jing Tang, and Zhijiang Guo. 2024b. Process-driven autoformalization in lean 4 . <i>Preprint</i> , arXiv:2406.01940.	797
748		798
749		799
750		800
751		801
752	John McCarthy. 2022. Artificial intelligence, logic, and formalising common sense. <i>Machine Learning and the City: Applications in Architecture and Urban Design</i> , pages 69–90.	802
753		803
754		804
755		
756	Jordan Meadows and André Freitas. 2021. Similarity-based equational inference in physics. <i>Physical Review Research</i> , 3(4):L042010.	805
757		806
758		807
759	Jordan Meadows, Tamsin Emily James, and Andre Freitas. 2024. Exploring the limits of fine-grained LLM-based physics inference via premise removal interventions . In <i>Findings of the Association for Computational Linguistics: EMNLP 2024</i> , pages 6487–6502, Miami, Florida, USA. Association for Computational Linguistics.	808
760		809
761		810
762		811
763		
764		
765		
766	Jordan Meadows, Marco Valentino, and André Freitas. 2025. Controlling equational reasoning in large language models with prompt interventions. In <i>Proceedings of the AAAI Conference on Artificial Intelligence</i> , volume 39, pages 24858–24866.	812
767		813
768		814
769		815
770		816
771	Agnieszka Mensfelt, David Tena Cucala, Santiago Franco, Angeliki Koutsoukou-Argyaki, Vince Trecsenyi, and Kostas Stathis. 2025. Towards a common framework for autoformalization . <i>Preprint</i> , arXiv:2509.09810.	817
772		818
773		819
774		
775		
	Andreas Opedal, Haruki Shirakami, Bernhard Schölkopf, Abulhair Saparov, and Mrinmaya Sachan. 2024. Mathgap: Out-of-distribution evaluation on problems with arbitrarily complex proofs. <i>arXiv preprint arXiv:2410.13502</i> .	820
		821
		822
		823
		824
	OpenAI, :, Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K. Arora, Yu Bai, Bowen Baker, Haiming Bao, Boaz Barak, Ally Bennett, Tyler Bertao, Nivedita Brett, Eugene Brevdo, Greg Brockman, Sebastien Bubeck, and 108 others. 2025. gpt-oss-120b & gpt-oss-20b model card . <i>Preprint</i> , arXiv:2508.10925.	825
		826
		827
		828
	OpenAI. 2025. Gpt-5.1: A smarter, more conversational chatgpt . Large language model.	829
		830
		831
	Auguste Poiroux, Gail Weiss, Viktor Kunčák, and Antoine Bosselut. 2025. Reliable evaluation and benchmarks for statement autoformalization . In <i>Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing</i> , pages 17958–17980, Suzhou, China. Association for Computational Linguistics.	
	Xin Quan, Marco Valentino, Louise Dennis, and Andre Freitas. 2024a. Enhancing ethical explanations of large language models through iterative symbolic refinement . In <i>Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)</i> , pages 1–22, St. Julian’s, Malta. Association for Computational Linguistics.	
	Xin Quan, Marco Valentino, Louise A. Dennis, and Andre Freitas. 2024b. Verification and refinement of natural language explanations through LLM-symbolic theorem proving . In <i>Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing</i> , pages 2933–2958, Miami, Florida, USA. Association for Computational Linguistics.	
	Z. Z. Ren, Zhihong Shao, Junxiao Song, Huajian Xin, Haocheng Wang, Wanjia Zhao, Liyue Zhang, Zhe Fu, Qihao Zhu, Dejian Yang, Z. F. Wu, Zhibin Gou, Shiron Ma, Hongxuan Tang, Yuxuan Liu, Wenjun Gao, Daya Guo, and Chong Ruan. 2025. Deepseek-prover-v2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition . <i>Preprint</i> , arXiv:2504.21801.	
	Alessandro Stolfo, Zhijing Jin, Kumar Shridhar, Bernhard Schölkopf, and Mrinmaya Sachan. 2022. A causal framework to quantify the robustness of mathematical reasoning with language models. <i>arXiv preprint arXiv:2210.12023</i> .	
	Guillem Tarrach, Albert Q. Jiang, Daniel Raggi, Wenda Li, and Mateja Jamnik. 2024. More details, please: Improving autoformalization with more detailed proofs . In <i>AI for Math Workshop @ ICML 2024</i> .	
	TeichAI / LioTix. 2025. TeichAI/Qwen3-14B-Claude-Sonnet-4.5-Reasoning-Distill-GGUF . Model page on Hugging Face. Accessed: 2026-01-01.	

George Tsoukalas, Jasper Lee, John Jennings, Jimmy Xin, Michelle Ding, Michael Jennings, Amitayush Thakur, and Swarat Chaudhuri. 2024. Putnambench: Evaluating neural theorem-provers on the putnam mathematical competition . In <i>The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track</i> .	890
Haiming Wang, Mert Unsal, Xiaohan Lin, Mantas Baksys, Junqi Liu, Marco Dos Santos, Flood Sung, Marina Vinyes, Zhenzhe Ying, Zekai Zhu, Jianqiao Lu, Hugues de Saxcé, Bolton Bailey, Chendong Song, Chenjun Xiao, Dehao Zhang, Ebony Zhang, Frederick Pu, Han Zhu, and 21 others. 2025a. Kimina-prover preview: Towards large formal reasoning models with reinforcement learning . <i>Preprint</i> , arXiv:2504.11354.	891
Shaoqi Wang, Lu Yu, Siwei Lou, Feng Yan, Chunjie Yang, Qing Cui, and Jun Zhou. 2025b. Improving autoformalization using direct dependency retrieval . <i>Preprint</i> , arXiv:2511.11990.	892
Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024. Executable code actions elicit better llm agents. In <i>Forty-first International Conference on Machine Learning</i> .	893
Yuhuai Wu, Albert Qiaochu Jiang, Wenda Li, Markus Norman Rabe, Charles E Staats, Mateja Jamnik, and Christian Szegedy. 2022. Autoformalization with large language models . In <i>Advances in Neural Information Processing Systems</i> .	894
Huajian Xin, Luming Li, Xiaoran Jin, Jacques Fleuriot, and Wenda Li. 2025. Ape-bench i: Towards file-level automated proof engineering of formal math libraries . <i>Preprint</i> , arXiv:2504.19110.	895
Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. 2025. The ai scientist-v2: Workshop-level automated scientific discovery via agentic tree search. <i>arXiv preprint arXiv:2504.08066</i> .	896
An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025a. Qwen3 technical report . <i>Preprint</i> , arXiv:2505.09388.	897
Kaiyu Yang, Gabriel Poesia, Jingxuan He, Wenda Li, Kristin E. Lauter, Swarat Chaudhuri, and Dawn Song. 2025b. Position: Formal mathematical reasoning—a new frontier in AI . In <i>Forty-second International Conference on Machine Learning Position Paper Track</i> .	898
Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan J Prenger, and Animashree Anandkumar. 2023a. Leandojo: Theorem proving with retrieval-augmented language models . In <i>Advances in Neural Information Processing Systems</i> , volume 36, pages 21573–21612. Curran Associates, Inc.	899
Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. 2023b. Leandojo: Theorem proving with retrieval-augmented language models . <i>Preprint</i> , arXiv:2306.15626.	900
Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. In <i>The eleventh international conference on learning representations</i> .	901
Huaiyuan Ying, Zijian Wu, Yihan Geng, Jiayu Wang, Dahua Lin, and Kai Chen. 2024. Lean workbook: A large-scale lean problem set formalized from natural language math problems . In <i>Advances in Neural Information Processing Systems</i> , volume 37, pages 105848–105863. Curran Associates, Inc.	902
Zhouliang Yu, Ruotian Peng, Keyi Ding, Yizhe Li, Zhongyuan Peng, Minghao Liu, Yifan Zhang, Zheng Yuan, Huajian Xin, Wenhao Huang, Yandong Wen, Ge Zhang, and Weiyang Liu. 2025. Formalmath: Benchmarking formal mathematical reasoning of large language models . <i>Preprint</i> , arXiv:2505.02735.	903
Lan Zhang, Xin Quan, and Andre Freitas. 2024. Consistent autoformalization for constructing mathematical libraries . In <i>Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing</i> , pages 4020–4033, Miami, Florida, USA. Association for Computational Linguistics.	904
Lan Zhang, Marco Valentino, and Andre Freitas. 2025a. Autoformalization in the wild: Assessing LLMs on real-world mathematical definitions . In <i>Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing</i> , pages 1720–1738, Suzhou, China. Association for Computational Linguistics.	905
Lan Zhang, Marco Valentino, and Andre Freitas. 2025b. Beyond gold standards: Epistemic ensemble of llm judges for formal mathematical reasoning . <i>Preprint</i> , arXiv:2506.10903.	906
Lan Zhang, Marco Valentino, and Andre Freitas. 2025c. MASA: LLM-driven multi-agent systems for autoformalization . In <i>Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations</i> , pages 615–624, Suzhou, China. Association for Computational Linguistics.	907
Meiru Zhang, Philipp Borchert, Milan Gritta, and Gerasimos Lampouras. 2025d. Drift: Decompose, retrieve, illustrate, then formalize theorems . <i>Preprint</i> , arXiv:2510.10815.	908
Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. 2022. minif2f: a cross-system benchmark for formal olympiad-level mathematics . In <i>International Conference on Learning Representations</i> .	909

A Implementation of FormalScience pipeline to construct FormalPhysics

As described in Alg. 1, the FormalScience pipeline requires in-context examples of informal statements and proofs, and a larger collection of informal proofs we aim to formalise, generate informal statements for, and extend or otherwise alter (*e.g.* add annotations) guided by the few-shot examples.

Data preparation. In the case of the physics examples, we curate 5 such gold-standard statement-proof pairs (see Fig. 4) and randomly select 200 examples from a related dataset of derivations (Meadows et al., 2024). We randomise the 200 examples and group them into batches of 5. Each batch, and the few-shot examples, are now in the required format for input to FormalScience. This ultimately results in 40 few-shot prompts (generated automatically with a Python script) for the initial generation of expanded statements and informal answers (see Section 2) using an LLM.

Stage 1: Generating informal statements and (expanded) proofs. We used GPT-5.1 in thinking mode to generate an intermediate dataset comprising of an informal "question" (statement) and "answer" (proof) from the initial few-shot prompt template (Tab. 4). In particular, the answers were expanded to include NL step annotations in a significant number of cases. After each few-shot prompt, the human expert evaluated the alignment between statement and proof, or otherwise prompted GPT to improve the alignment. Each of the 40 resulting raw LLM outputs contained alternating questions (Q1 - Q5) and answers (A1 - A5). These were split up via a post-processing script to form an intermediate dataset of 40 examples where each example contains 5 dictionaries per derivation. Each dictionary contains the "field" (*e.g.* electromagnetism), the question, and the answer.

Stage 2: Generating formal proofs in Lean4. Each example from the intermediate dataset from the previous stage was input to a formalisation prompt template (see Tab 4).

Our implementation diverges at this point into two complimentary approaches, as we used the ChatGPT interface to produce around 1/3 of examples, and Claude Code (through VSCode) to generate the rest.

The ChatGPT approach required manually copying the outputted formal code, compiling it in Lean, then prompting GPT (within the same session) with

the raw compilation errors. We removed non-fatal warnings to reduce context window limitations. Per initial formalisation prompt, this approach required approximately 1-2 hours and several rounds of prompts to generate outputs without compilation errors. Occasionally GPT had to be reminded that incorrect imports did not mean the Lean environment was incomplete.

The Claude Code approach (recommended) included less manual compilation and human intervention. A custom Lean compilation (Python) script was written (*i.e.* $\mathcal{L}(C)$, Section 2), and instructions on how to use it, and handle its output, were appended to the end of the formalisation prompt template. Importantly, due to the context window compactification implemented frequently during this approach, we also saved the exact formalisation prompt in a separate text file in the same folder as the Lean compilation tool, which we found aided the later alignment stage.

Each generated formal code output is prefixed with "C1-C5", due to the formalisation prompt template containing questions "Q1-Q5" and informal solutions "A1-A5".

Stage 3: Iterative alignment. Upon successful compilation without errors, our approach to iterative alignment was centered around a single fixed prompt:

"How well do C1-C5 align with A1-A5, the Requirements, and the Acceptance criteria?"

In either the Claude Code or ChatGPT interface approaches, the human expert evaluates whether the resulting alignment analysis is acceptable. Reaching this point in the pipeline is in itself an iterative process we call *patience*, $\mathcal{P}$, in Alg. 1. We used a maximum patience $\mathcal{P} = 3$ before considering the formal code "well-aligned".

If the formal code was not aligned, an additional fixed prompt (*"Make the suggested improvements and ensure C1-C5 aligns with A1-A5, the Requirements, and the Acceptance criteria."*) was used in either case. Claude Code had to additionally be instructed to iteratively use the Lean compilation tool, while this was manually achieved by the human expert for ChatGPT. This essentially restarts Stage 2 and ticks patience $\mathcal{P}$. Notably, for GPT, we found only compiling the poorly-aligned code vastly accelerated this process.

Stage 4: Post-processing and formal re-verification. Both approaches converge to the same post-processing method. Each formal code

Q2: Consider a time-independent self-adjoint Hamiltonian $\hat{H}_\lambda$ that smoothly depends on a real parameter λ , with non-degenerate eigenvalue E_λ and eigenvector $|\Psi_\lambda\rangle$, such that Schrodinger holds, i.e.

$$\hat{H}_\lambda |\Psi_\lambda\rangle = E_\lambda |\Psi_\lambda\rangle$$

Prove that

$$\frac{\partial E_\lambda}{\partial \lambda} = E_\lambda \left(\frac{\partial \Psi_\lambda}{\partial \lambda} | \Psi_\lambda \right) + \langle \Psi_\lambda | \frac{\partial \hat{H}_\lambda}{\partial \lambda} | \Psi_\lambda \rangle + E_\lambda \langle \Psi_\lambda | \frac{\partial \Psi_\lambda}{\partial \lambda} \rangle$$

A2: If we take the inner product of each side of the Schrodinger equation, we obtain

$$\langle \Psi_\lambda | \hat{H}_\lambda | \Psi_\lambda \rangle = E_\lambda \langle \Psi_\lambda | \Psi_\lambda \rangle$$

As states are assumed normalized, we recall that $\langle \Psi_\lambda | \Psi_\lambda \rangle = 1$, and hence

$$E_\lambda = \langle \Psi_\lambda | \hat{H}_\lambda | \Psi_\lambda \rangle$$

If we take the partial derivative with respect to λ we obtain

$$\frac{\partial E_\lambda}{\partial \lambda} = \frac{\partial}{\partial \lambda} \langle \Psi_\lambda | \hat{H}_\lambda | \Psi_\lambda \rangle$$

Next, via application of the chain rule, we can write

$$\frac{\partial E_\lambda}{\partial \lambda} = \left(\frac{\partial \Psi_\lambda}{\partial \lambda} | \hat{H}_\lambda | \Psi_\lambda \right) + \langle \Psi_\lambda | \frac{\partial \hat{H}_\lambda}{\partial \lambda} | \Psi_\lambda \rangle + \langle \Psi_\lambda | \hat{H}_\lambda | \frac{\partial \Psi_\lambda}{\partial \lambda} \rangle$$

Finally, by substituting in the RHS of the Schrodinger equation and factoring out E_λ , we obtain

$$\frac{\partial E_\lambda}{\partial \lambda} = E_\lambda \left(\frac{\partial \Psi_\lambda}{\partial \lambda} | \Psi_\lambda \right) + \langle \Psi_\lambda | \frac{\partial \hat{H}_\lambda}{\partial \lambda} | \Psi_\lambda \rangle + E_\lambda \langle \Psi_\lambda | \frac{\partial \Psi_\lambda}{\partial \lambda} \rangle$$

Figure 4: A (human-written) in-context example used in a few-shot prompt to automatically provide physical context to other examples with a similar degree of depth.

output from Stage 3 (a total of 40) contains 5 separate formal code proofs C1-C5, where Mathlib imports for all proofs are combined at the top of the file, within the same block. We use a Python script to separate out all proofs into 5 separate files with identical import blocks. We use another script to unify all questions, informal answers, formal proofs, and physics subdomain categories into a dataset of 200 examples.

We compile each formal proof in this dataset to determine if the separation script introduced any errors, making a list of example IDs with new errors. We iteratively improve each proof, beginning from Stage 3 in each case, but reword the alignment prompts to consider only one input question (Q), informal answer (A), and formal code (C) at a time. The resulting examples are added back into the dataset, which finalises our implementation of the FormalScience pipeline used to generate the FormalPhysics dataset.

B Prompts

We provide the few-shot template, the prompt for FormalScience, and prompt for testing LLMs in Figure 4, Table 4, Table 5, respectively.

C Derivation of the agentic code generation pipeline

Let T_g be the initial prompt template for physics question x . The initial generation phase attempts

up to $N = 25$ iterations:

$$C^{(0)} = \text{extract}(\mathcal{M}(T_g(x))) \quad (6)$$

where $\text{extract} : \Sigma^* \rightarrow \text{Code}$ removes markdown fences from the LLM output. Each candidate $C^{(i)}$ is passed through a surface guard $\mathcal{G} : \text{Code} \rightarrow \{0, 1\} \times \Sigma^*$ which performs syntax validation before compiling in Lean. The guard returns $(1, \varepsilon)$ if the code passes all heuristic checks, or $(0, r)$ with rejection reason r otherwise. These checks enforce constraints already specified in T_g but which the LLM may violate: absence of forbidden tokens $\mathcal{F} = \{\partial, \nabla, \dot{x}, \dot{y}, \dot{z}, \dagger, \backslash\backslash, '\}$, absence of incomplete proof markers (sorry, axiom), balanced comment delimiters, and correct import ordering. If the guard fails, the agent regenerates with feedback:

$$C^{(i+1)} = \text{extract}(\mathcal{M}(T_r(x, r^{(i)}, \perp))) \quad (7)$$

where $\mathcal{G}(C^{(i)}) = (0, r^{(i)})$, the T_r is a regeneration template that appends the rejection reason to the base prompt, and $\perp$ denotes the absence of a compiler error (since compilation has not yet been attempted). The phase terminates at $i^* = \min\{i : \mathcal{G}(C^{(i)}) = (1, \varepsilon)\}$ or fails if no valid candidate is found within N attempts.

Once initial generation succeeds, the agent enters an iterative compilation loop for up to $N_{\max} = 25$ steps. At each step n , the Lean compiler $\mathcal{L}(C^{(n)})$ returns $(0, \varepsilon)$ on successful compilation or $(1, e)$ with error message e otherwise (i.e. Eq. 2). When compilation fails, a categorisation function $\kappa : \Sigma^* \rightarrow \mathcal{E}$ maps the error to one of six categories via pattern matching $\mathcal{E} = \{\text{syntax}, \text{unknown_id}, \text{missing_module}, \text{type_mismatch}, \text{unsolved_goals}, \text{other}\}$. These categories are split into structural errors $\mathcal{E}_{\text{struct}} = \{\text{syntax}, \text{unknown_id}, \text{missing_module}\}$ and semantic errors $\mathcal{E}_{\text{sem}} = \{\text{type_mismatch}, \text{unsolved_goals}, \text{other}\}$, which determine the repair strategy.

For structural errors, the agent performs full regeneration using the primary LLM:

$$C^{(t+1)} = \text{extract}(\mathcal{M}(T_r(x, \kappa(e), e))) \quad (8)$$

if $\kappa(e) \in \mathcal{E}_{\text{struct}}$ where T_r now includes both the error category and the compiler message,

Algorithm 1 FormalScience

Require: Derivation batches $\mathcal{D}$, batch size B (with $B' \in [1, B]$), gold-standard examples $\mathcal{D}^* = [(Q_i, A_i)]_{i=1}^{N'}$, patience $\mathcal{P}$. Here, questions Q and informal answers A are respectively equivalent to informal statements S and informal proofs P in the main text.

Ensure: Formally verified corpus $Z = \{(Q, A, C) : \mathcal{L}(C) = (0, \varepsilon)\}$

Stage 1: Informal QA Generation and Alignment

```
1:  $X \leftarrow \emptyset$ 
2: for each batch  $d \in \mathcal{D}$  do
3:    $x \leftarrow S(\mathcal{M}(T_{fs}(d, \mathcal{D}^*); P_a))$  ▷ Few-shot QA generation with human alignment
4:    $X \leftarrow X \cup \{x\}$ 
5: end for
```

Stage 2: Code Generation and Iterative Correction

```
6:  $Y \leftarrow \emptyset$ 
7: for each  $x = [(Q_i, A_i)]_{i=1}^{B'} \in X$  do
8:    $C^{(0)} \leftarrow \mathcal{M}'(T_g(x))$  ▷ Initial code generation
9:    $t \leftarrow 0$ 
10:  while  $\mathcal{L}(C^{(t)}) = (1, e)$  do ▷ Compilation fails
11:     $C^{(t+1)} \leftarrow \mathcal{M}'(T_c(x, C^{(t)}, e))$  ▷ Error-guided correction
12:     $t \leftarrow t + 1$ 
13:  end while
14:   $C \leftarrow C^{(t)}$  ▷  $C = \mathcal{R}(C^{(0)})$ 
```

Stage 3: Formal Language Alignment

```
15:  $k \leftarrow 0$ 
16: while  $\mathcal{H}^{(k)} = 1$  and  $k < \mathcal{P}$  do ▷ Human rejects alignment
17:    $C' \leftarrow \mathcal{M}'(\mathcal{M}'(T_g(x), C^{(k)}), P_g)$  ▷ Alignment improvement
18:    $C^{(k+1)} \leftarrow \mathcal{R}(C')$  ▷ Re-verify compilation
19:    $k \leftarrow k + 1$ 
20: end while
21:  $y \leftarrow S'(x, C^{(k)})$  ▷ Split into  $(Q_i, A_i, C_i)$  tuples
22:  $Y \leftarrow Y \cup \{y\}$ 
23: end for
```

Stage 4: Post-processing and Final Verification

```
24:  $Z \leftarrow \text{flatten}(Y)$ 
25: for each  $(Q, A, C) \in Z$  do
26:   if  $\mathcal{L}(C) = (1, e)$  then ▷ Post-processing introduced errors
27:      $(Q, A, C) \leftarrow \text{apply Eq. 5}$  ▷ Re-align and correct
28:   end if
29: end for
30: return  $Z$ 
```

1119 instructing the model to rewrite the entire file.
1120 The regenerated code must again pass the surface
1121 guard; if $\mathcal{G}(C^{(t+1)}) = (0, _)$, the regeneration is
1122 discarded and the step is logged as failed.
1123

patch agent $\mathcal{A}_{\text{patch}}$ equipped with a unified diff tool:

$$C^{(t+1)} = \mathcal{A}_{\text{patch}}(T_p(\text{number}(C^{(t)}), e)) \quad (9)$$

1127 if $\kappa(e) \in \mathcal{E}_{\text{sem}}$ where $\text{number} : \text{Code} \rightarrow \text{Code}$
1128 prepends line numbers and T_p is a patch template
1129 requesting a minimal unified diff. The patch agent
1130 operates in a ReAct loop, generating a diff and

1124 For semantic errors, the agent uses a specialised

Prompt Name	Purpose	Template Structure	Input Variables	Output Expected
Informal Expansion Prompt	Convert equation-only derivations into contextually-rich Q&A pairs	<i>“The following 5 questions (Q1–Q5) and respective answers (A1–A5) are few-shot examples...”</i> followed by 5 quantum Q&A pairs, then <i>“Now, the following equation-only derivations (D6–D10) represent the underlying equational reasoning of a Physics derivation. You must convert each derivation into a physically-correct and contextually-enriched Question (Q6–Q10) and Answer (A6–A10) pair...”</i> followed by 5 derivations and closing instruction to ensure one equality per equation with correct physical meaning and standard notation.	• 5 few-shot Q&A pairs. • 5 equation-chain derivations (<i>i.e.</i> a batch).	5 new Q&A pairs (Q6–Q10, A6–A10) with physics context and properly formatted $\text{\LaTeX}$ equations
Lean Formalization Prompt	Autoformalize informal physics derivations into compilable Lean 4 + Mathlib proofs	Header: Task description requesting compilable Lean 4 proofs without axioms. Q/A Block: 5 pairs formatted as “<i>Qi: {question} Ai: {answer}</i>”. Requirements: (1) No axiom/sorry; (2) Use Mathlib theorems; (3) Explicit physics modelling; (4) Single compilable file; (5) Include docstrings and clear theorem names; (6) Prefer calc blocks over simp; (7) Deterministic rewrites. Deliverables: One file with C1–C5. Acceptance: Must compile with no axioms.	• 5 expanded informal Q&A pairs from previous LLM output.	Single Lean 4 file containing 5 theorems (C1–C5) with Mathlib imports, docstrings, explicit hypotheses, and complete proofs

Table 4: Prompt Templates Used in the FormalScience Pipeline

applying it via the `apply_unified_diff` tool. This reflects the intuition that structural errors indicate fundamental misunderstanding requiring complete regeneration, while semantic errors (type mismatches, unsolved goals) are often addressable through localised edits to tactics or expressions.

The correction loop terminates when $\mathcal{L}(C^{(t)}) = (0, \varepsilon)$ or when $n = N_{\max}$. Unlike the human-in-the-loop pipeline (Alg. 1, Stage 3), this approach does not perform formal language alignment. The agentic code generation pipeline used to obtain the relevant baselines is described in Alg. 2.

D Supplementary Analysis

Task	Content
Zero-Shot Statement Autoformalisation	You are an expert in formal language Lean4. You will be given a physics statement and its proof written in natural language and LaTeX symbols. Your task is to provide the formal code of the given natural language physics statement and its proof in Lean4 with the following instructions: 1. You should give the formal code directly without any additional comments or explanations. If the given physics statement is a theorem or lemma, omit the formal proof and use the default 'sorry' mode in the formal code. 2. In case that you need to import any necessary preambles, you should not import any fake (non-exist) preambles. 3. You should wrap the formal code in a way illustrated as the following: <pre>%%%%%%%%%%%%%% Your Formal Code %%%%%%%%%%%%%%</pre> Strictly follow the instructions that have been claimed. Natural language statement: {nl_statement} Give me the Lean4 formal code of the statement:
Self-Refinement with Error Feedback	You are an expert in formal language Lean4. You will be given a physics statement and its proof written in natural language and LaTeX symbols. You will also be given a formal code which attempted to describe the given physics statement in Lean4. Your task is to refine the given formal code to make it correct while maintaining the alignment with the given natural language physics statement. Here are some instructions for your task: 1. You should give the formal code directly without any additional comments or explanations. If the given physics statement is a theorem or lemma, omit the formal proof and use the default 'sorry' mode in the formal code. 2. In case that you need to import any necessary preambles, you should not import any fake (non-exist) preambles. 3. You should wrap the formal code in a way illustrated as the following: <pre>%%%%%%%%%%%%%% Your Formal Code %%%%%%%%%%%%%%</pre> Strictly follow the instructions that have been claimed. Natural language statement: {nl_statement} There are some Lean4 formal codes describing the given physics statement: {formal} You should refine the formal code for your task to make it correct. Here are some feedbacks about the formal code which can be used to help your task: {According to the theorem prover, the error details of the provided formal code are: error_details }

Table 5: Prompts.

Algorithm 2 Agentic Code Generation Pipeline

Require: Physics question x , initial generation template T_g , regeneration template T_r , patch template T_p , max initial attempts $N = 25$, max correction steps $N_{\max} = 25$, forbidden tokens $F = \{\partial, \nabla, \dot{x}, \dot{y}, \dot{z}, \dagger, \backslash, ', \}$

Ensure: Compilable code C such that $\mathcal{L}(C) = (0, \varepsilon)$, or failure

Stage 1: Initial Generation with Surface Guard

```
1:  $i \leftarrow 0$ 
2: while  $i < N$  do
3:    $C^{(i)} \leftarrow \text{extract}(\mathcal{M}(T_g(x)))$  ▷ Eq. 9: Generate and extract code
4:    $(g, r^{(i)}) \leftarrow G(C^{(i)})$  ▷ Surface guard validation
5:   if  $g = 1$  then ▷ Guard passed
6:     goto Stage 2
7:   else
8:      $C^{(i+1)} \leftarrow \text{extract}(\mathcal{M}(T_r(x, r^{(i)}, \perp)))$  ▷ Eq. 10: Regenerate with feedback
9:   end if
10:   $i \leftarrow i + 1$ 
11: end while
12: return FAILURE ▷ No valid candidate in  $N$  attempts
```

Stage 2: Iterative Compilation and Error Correction

```
13:  $C \leftarrow C^{(i^*)}$  where  $i^* = \min\{i : G(C^{(i)}) = (1, \varepsilon)\}$ 
14:  $t \leftarrow 0$ 
15: while  $t < N_{\max}$  do
16:    $(s, e) \leftarrow \mathcal{L}(C^{(t)})$  ▷ Eq. 5: Lean compilation
17:   if  $s = 0$  then ▷ Compilation succeeded
18:     return  $C^{(t)}$ 
19:   end if
20:    $\kappa(e) \leftarrow \text{CATEGORIZE}(e)$  ▷ Error categorisation
21:   if  $\kappa(e) \in E_{\text{struct}}$  then ▷ Structural errors: syntax, unknown_id, missing_module
22:      $C' \leftarrow \text{extract}(\mathcal{M}(T_r(x, \kappa(e), e)))$  ▷ Eq. 11: Full regeneration
23:     if  $G(C') = (1, \varepsilon)$  then ▷ Guard passes
24:        $C^{(t+1)} \leftarrow C'$ 
25:     else
26:       Log regeneration failure;  $C^{(t+1)} \leftarrow C^{(t)}$ 
27:     end if
28:   else ▷ Semantic errors: type_mismatch, unsolved_goals, other
29:      $C^{(t+1)} \leftarrow \mathcal{A}_{\text{patch}}(T_p(\text{numberlines}(C^{(t)}), e))$  ▷ Eq. 12: Patch agent
30:   end if
31:    $t \leftarrow t + 1$ 
32: end while
33: return  $C^{(N_{\max})}$  ▷ Return best effort after max steps
```

– Helper Definitions –

```
34: function CATEGORIZE( $e$ )
35:    $E_{\text{struct}} \leftarrow \{\text{syntax, unknown\_id, missing\_module}\}$ 
36:    $E_{\text{sem}} \leftarrow \{\text{type\_mismatch, unsolved\_goals, other}\}$ 
37:   return category  $\kappa(e) \in E_{\text{struct}} \cup E_{\text{sem}}$  via pattern matching
38: end function
```

```
39: function  $G(C)$  ▷ Surface guard:  $G : \text{Code} \rightarrow \{0, 1\} \times \Sigma^*$ 
40:   if  $\exists f \in F : f \in C$  then return  $(0, \text{"forbidden token " } f)$ 
41:   end if
42:   if  $\text{sorry} \in C \vee \text{axiom} \in C$  then return  $(0, \text{"incomplete proof"})$ 
43:   end if
44:   if  $|C|_{/-} \neq |C|_{-/}$  then return  $(0, \text{"unmatched delimiters"})$ 
45:   end if
46:   if imports not correctly ordered then return  $(0, \text{"import ordering"})$ 
47:   end if
48:   return  $(1, \varepsilon)$ 
49: end function
```

Table 6: Detailed semantic drift analysis: Physics notation versus Lean4 formalisation. This table provides extended examples showing how mathematical objects transform during autoformalisation.

ID	Physics (Informal)	Lean4 (Formal)
Pattern A: Quantum Operators $\rightarrow$ Scalars/Algebra Elements		
4	Statement: $N = a^\dagger a = \frac{1}{2}(q - ip)(q + ip)$ Key objects: q, p: Position/momentum operators with $[q, p] = i$ $a, a^\dagger$: Ladder operators (non-commuting) N: Number operator Physics content: Operator algebra on Hilbert space	<pre>theorem C5_number_operator_expand (q p : C) : -- Complex numbers! let s := (1 : R) / Real.sqrt 2 let a := s * (q + I * p) let adag := s * (q - I * p) adag * a = s*s * (p^2 - I*(p*q) + I*(q*p) + q^2)</pre> Drift: $[q, p] = i$ not enforced; proof holds for any complex numbers
14	Statement: Heisenberg uncertainty $\Delta X \cdot \Delta P \geq \hbar/2$ Key objects: $f\rangle, g\rangle$: Fluctuation kets in Hilbert space $[\hat{X}, \hat{P}] = i\hbar$: Canonical commutation relation - Cauchy-Schwarz on inner products Physics content: Fundamental quantum bound	<pre>theorem C1 {E : Type*} [InnerProductSpace C E] {f g : E} {hbar : R} (hbar_nonneg : 0 <= hbar) (hcomm : inner f g - inner g f = I * (hbar : C)) : f * g >= hbar / 2</pre> Drift: Commutator relation is a <i>hypothesis</i>, not derived from $[\hat{X}, \hat{P}]$
12	Statement: $\frac{d\hat{x}(t)}{dt} = \frac{i}{\hbar}[\hat{H}, \hat{x}(t)]$ Key objects: $e^{i\hat{H}t/\hbar}$: Unitary time evolution operator $\hat{x}(t) = U(t)\hat{x}U^\dagger(t)$: Heisenberg picture - Product rule on operator exponentials Physics content: Quantum dynamics	<pre>theorem C3 {A : Type*} [NormedAlgebra R A] (c : R) {H x0 : A} (U W : R -> A) (hU : forall t, deriv U t = c*(H*U t)) (hW : forall t, deriv W t = -c*(W t*H)) (hUd : forall t, DifferentiableAt R U t) (hWd : forall t, DifferentiableAt R W t) : forall t, deriv (s => U s * x0 * W s) t = c * (H*(U t*x0*W t) - (U t*x0*W t)*H)</pre> Drift: Exponential structure assumed via hU, hW; works in any normed algebra
Pattern B: Vector Calculus $\rightarrow$ Abstract Linear Maps		
5	Statement: $\hat{\mathbf{r}} \cdot \nabla f = \frac{\partial f}{\partial r}$ Key objects: $\nabla f = \frac{\partial f}{\partial r}\hat{\mathbf{r}} + \frac{1}{r}\frac{\partial f}{\partial \theta}\hat{\boldsymbol{\theta}} + \frac{1}{r\sin\theta}\frac{\partial f}{\partial \varphi}\hat{\boldsymbol{\varphi}}$ - Spherical coordinate basis $\{\hat{\mathbf{r}}, \hat{\boldsymbol{\theta}}, \hat{\boldsymbol{\varphi}}\}$ Physics content: Coordinate geometry	<pre>theorem C1_radial_projection {E : Type*} [InnerProductSpace R E] {er e_th e_ph : E} {ar ath aph : R} (hnorm : er = 1) (horth_r_th : inner er e_th = 0) (horth_r_ph : inner er e_ph = 0) : inner er (ar*er + ath*e_th + aph*e_ph) = ar</pre> Drift: Gradient formula becomes hypothesis; spherical coords $\rightarrow$ abstract orthonormal basis
24	Statement: Gauss's law from $\nabla^2 \phi = 4\pi G\rho$ Key objects: $\int_V \nabla^2 \phi dV = 4\pi G \int_V \rho dV$ - Divergence theorem: $\int_V \nabla \cdot \mathbf{F} dV = \oint_{\partial V} \mathbf{F} \cdot d\mathbf{S}$ Physics content: Field theory	<pre>theorem C5_gauss_from_poisson (G : R) (S V1 V2 : R) (hPoisson : V1 = (4 * pi * G) * V2) (hDivThm : S = V1) : S = (4 * pi * G) * V2 := by simp [hPoisson] using hDivThm</pre> Drift: All integration/divergence content in hypotheses; proof is trivial substitution
Pattern C: Integration $\rightarrow$ Limit/Algebraic Statement		
10	Statement: $W = \int_{r_0}^\infty \frac{GMm}{r^2} dr = \frac{GMm}{r_0}$ Key objects: - Definite integral of inverse-square force - Improper integral at infinity Physics content: Work-energy theorem	<pre>theorem C1 (G M m r0 : R) (hr0 : 0 < r0) : Tendsto (fun R => (G*M*m)*(r0^-1 - R^-1)) atTop (nhds ((G*M*m) / r0)) := by have h_inv : Tendsto (R^-1) atTop (nhds 0) := tendsto_inv_atTop_zero ...</pre> Drift: Integral $\rightarrow$ limit statement; uses tendsto_inv_atTop_zero lemma
27	Statement: $U = q\phi(r)$ from $W = \int_\infty^r \mathbf{F} \cdot d\mathbf{l}$ Key objects: - Line integral of electric force - Reference $\phi(\infty) = 0$ Physics content: Potential energy definition	<pre>theorem C3_potential_energy_from_reference (q r : R) (phi : R -> R) : q * (phi r - 0) = q * phi r := by simp [sub_zero]</pre> Drift: Entire integration content abstracted; proof is sub_zero